

Big Moves on Monday (Long Only)

The strategy suggests that we go long on the S&P 500 on a Monday and close out our position on the Friday week based on certain conditions that positions can be taken only in periods when markets are open from Monday to Friday in a week and the Friday in the Previous week.

We calculate the following indicator and backtest the conditions shown below :

1. Calculate the 25 day average of $\text{relative_range} = (\text{High} - \text{low}) / \text{close}$ and call it `rel_range_ma`
2. The Monday close must be lower than the previous Friday close by atleast 0.25 times of `rel_range_ma`.
3. Create a variable `ibs = (\text{close} - \text{low}) / (\text{high} - \text{low})`. It must be lower than 0.3
4. if condition in 2 and 3 are met go long on Monday Close
5. Square off your position on Friday Close.

```
In [30]: import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
import datetime
```

```
In [25]: def download_daily_data(ticker,start,end):
        """
        The function downloads daily market data to pandas DataFrame
        using the 'yfinance' API between the dates specified.
        """
        data = yf.download(ticker,start,end, auto_adjust=False)
        return data
```

```
In [55]: def compute_daily_returns(data):
        """
        The functions computes daily log returns on the close Prices.
        """
        data["CC_returns"] = np.log(data["Close"]/data["Close"].shift(1))
        return data
```

```
In [56]: ticker1 = "SPY"
end1 = datetime.date(2020,7,2)
start1 = end1 - pd.Timedelta(days=365*15)
```

```
In [97]: start1
```

```
Out[97]: datetime.date(2005, 7, 6)
```

```
In [98]: df = download_daily_data(ticker=ticker1, start=start1,end=end1)
```

[*****100%*****] 1 of 1 completed

In [99]: df

Out[99]:

	Price	Adj Close	Close	High	Low	Open	Volume
	Ticker	SPY	SPY	SPY	SPY	SPY	SPY
	Date						
	2005-07-06	81.264610	119.480003	120.650002	119.410004	120.389999	52363600
	2005-07-07	81.584290	119.949997	119.949997	118.260002	118.290001	103268800
	2005-07-08	82.516106	121.320000	121.320000	119.720001	119.970001	64491200
	2005-07-11	82.937805	121.940002	122.099998	121.309998	121.330002	49688300
	2005-07-12	83.155464	122.260002	122.629997	121.639999	121.989998	51871100
	...	...	...	...	...	...	...
	2020-06-25	283.042938	307.350006	307.640015	301.279999	303.470001	89468000
	2020-06-26	276.320251	300.049988	306.390015	299.420013	306.160004	127961000
	2020-06-29	280.381531	304.459991	304.609985	298.929993	301.410004	79773300
	2020-06-30	283.973022	308.359985	310.200012	303.820007	303.989990	113394800
	2020-07-01	285.962219	310.519989	311.890015	309.070007	309.570007	72396500

3774 rows × 6 columns

In [87]: df1 = df.copy()

In [88]: df1 =compute_daily_returns(data=df1)

In [89]: df1

Out[89]:

Price	Adj Close	Close	High	Low	Open	Volume	CC_returns
Ticker	SPY	SPY	SPY	SPY	SPY	SPY	
Date							
2005-07-06	81.264610	119.480003	120.650002	119.410004	120.389999	52363600	NaN
2005-07-07	81.584290	119.949997	119.949997	118.260002	118.290001	103268800	0.003926
2005-07-08	82.516106	121.320000	121.320000	119.720001	119.970001	64491200	0.011357
2005-07-11	82.937805	121.940002	122.099998	121.309998	121.330002	49688300	0.005097
2005-07-12	83.155464	122.260002	122.629997	121.639999	121.989998	51871100	0.002621
...	...	...	...	...	...	...	...
2020-06-25	283.042938	307.350006	307.640015	301.279999	303.470001	89468000	0.010663
2020-06-26	276.320251	300.049988	306.390015	299.420013	306.160004	127961000	-0.024038
2020-06-29	280.381531	304.459991	304.609985	298.929993	301.410004	79773300	0.014591
2020-06-30	283.973022	308.359985	310.200012	303.820007	303.989990	113394800	0.012728
2020-07-01	285.962219	310.519989	311.890015	309.070007	309.570007	72396500	0.006980

3774 rows × 7 columns

```

In [90]: def compute_indicators(data):
    ### columns created to check condition 1
    data['day'] = data.index.day_name()
    data["previous_day"] = data['day'].shift(1)
    data["Four_days_after"] = data['day'].shift(-4)

    ### columns created to check condition 2
    data["relative_range"] = (data["High"] - data["Low"])/data["Close"]
    data["rel_range_ma"] = data["relative_range"].rolling(window=25).mean()

    ### columns to check condition 3
    data["ibs"] = (data["Close"] - data["Low"])/(data["High"] - data["Low"])

    return data

```

```
In [91]: def backtest_strategy(data):

    data["Condition1"] = np.where((data["day"] == "Monday")
                                   & (data["previous_day"] == "Friday")
                                   & (data["Four_days_after"] == "Friday"),
                                   1,0)

    close_series = data["Close"].squeeze()
    data["Condition2"] = np.where((1 - close_series / close_series.shift(1)) >= 0.2,
                                   1,0)

    data["Condition3"] = np.where(data["ibs"] < 0.3, 1, 0)

    data["Signal"] = np.where((data["Condition1"] == 1)
                              & (data["Condition2"] == 1)
                              & (data["Condition3"] == 1),
                              1,0)

    data["Signal"] = data["Signal"].shift(1)
    data["position"] = data["Signal"].replace(to_replace=0, method="ffill", limit=1)
    data["Strategy_returns"] = data["CC_returns"] * data["position"]

    return data
```

```
In [92]: compute_indicators(df1)
```

Out[92]:

Price	Adj Close	Close	High	Low	Open	Volume	CC_returns
Ticker	SPY	SPY	SPY	SPY	SPY	SPY	
Date							
2005-07-06	81.264610	119.480003	120.650002	119.410004	120.389999	52363600	NaN
2005-07-07	81.584290	119.949997	119.949997	118.260002	118.290001	103268800	0.003926
2005-07-08	82.516106	121.320000	121.320000	119.720001	119.970001	64491200	0.011357
2005-07-11	82.937805	121.940002	122.099998	121.309998	121.330002	49688300	0.005097
2005-07-12	83.155464	122.260002	122.629997	121.639999	121.989998	51871100	0.002621
...	...	...	...	...	...	...	...
2020-06-25	283.042938	307.350006	307.640015	301.279999	303.470001	89468000	0.010663
2020-06-26	276.320251	300.049988	306.390015	299.420013	306.160004	127961000	-0.024038
2020-06-29	280.381531	304.459991	304.609985	298.929993	301.410004	79773300	0.014591
2020-06-30	283.973022	308.359985	310.200012	303.820007	303.989990	113394800	0.012728
2020-07-01	285.962219	310.519989	311.890015	309.070007	309.570007	72396500	0.006980

3774 rows × 13 columns



In [93]:

backtest_strategy(df1)

C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\2761979206.py:21: FutureWarning: The 'method' keyword in Series.replace is deprecated and will be removed in a future version.
data["position"] = data["Signal"].replace(to_replace=0, method="ffill", limit = 3)

Out[93]:

Price	Adj Close	Close	High	Low	Open	Volume	CC_returns
Ticker	SPY	SPY	SPY	SPY	SPY	SPY	
Date							
2005-07-06	81.264610	119.480003	120.650002	119.410004	120.389999	52363600	NaN
2005-07-07	81.584290	119.949997	119.949997	118.260002	118.290001	103268800	0.003926
2005-07-08	82.516106	121.320000	121.320000	119.720001	119.970001	64491200	0.011357
2005-07-11	82.937805	121.940002	122.099998	121.309998	121.330002	49688300	0.005097
2005-07-12	83.155464	122.260002	122.629997	121.639999	121.989998	51871100	0.002621
...	...	...	...	...	...	...	...
2020-06-25	283.042938	307.350006	307.640015	301.279999	303.470001	89468000	0.010663
2020-06-26	276.320251	300.049988	306.390015	299.420013	306.160004	127961000	-0.024038
2020-06-29	280.381531	304.459991	304.609985	298.929993	301.410004	79773300	0.014591
2020-06-30	283.973022	308.359985	310.200012	303.820007	303.989990	113394800	0.012728
2020-07-01	285.962219	310.519989	311.890015	309.070007	309.570007	72396500	0.006980

3774 rows × 19 columns



In [94]:

```
def show_backtesting_results(data):
    print("Buy and Hold returns : ", np.round(data["CC_returns"].cumsum()[-1],2))
    print("Strategy_returns : " , np.round(data["Strategy_returns"].cumsum()[-1],2))

    data[["CC_returns", "Strategy_returns" ]] = data[["CC_returns", "Strategy_returns", "position"]].plot(grid=True)
```

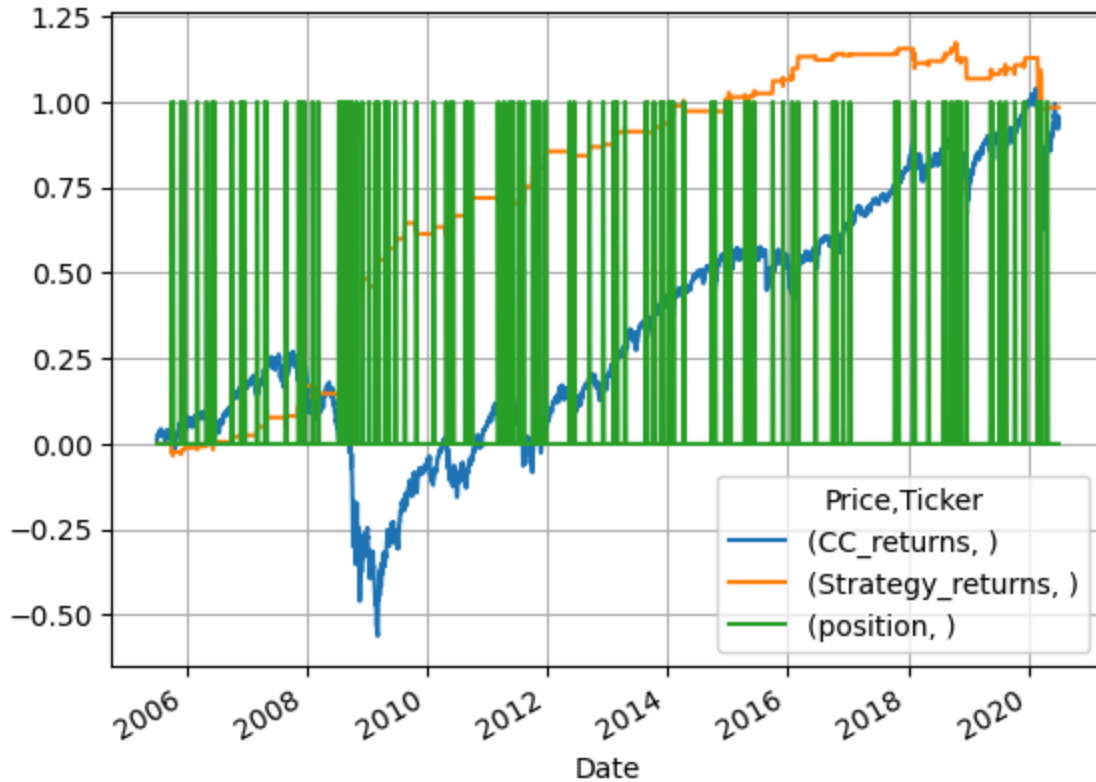
In [95]:

```
show_backtesting_results(df1)
```

```

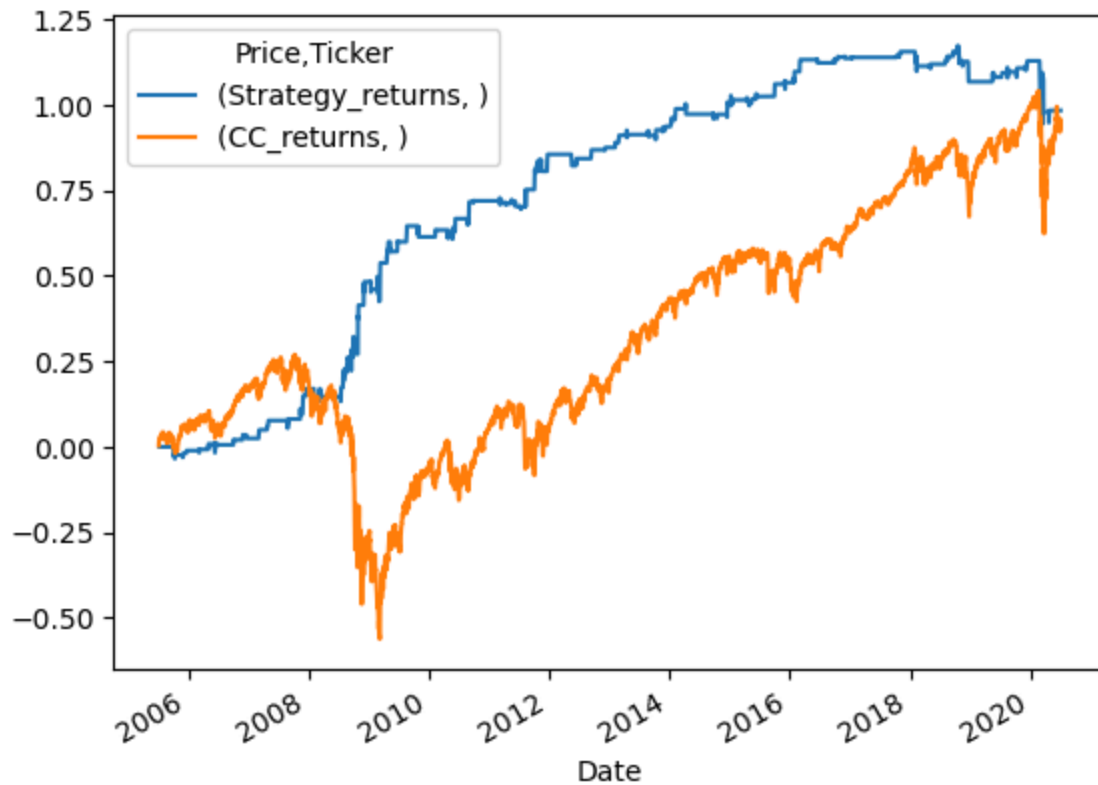
C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\714521281.py:3: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    print("Buy and Hold returns : ", np.round(data["CC_returns"].cumsum()[-1],2))
C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\714521281.py:4: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
    print("Strategy_returns : " , np.round(data["Strategy_returns"].cumsum()[-1],2))
Buy and Hold returns : 0.96
Strategy_returns : 0.98

```



```
In [96]: df1[["Strategy_returns", "CC_returns"]].plot()
```

```
Out[96]: <Axes: xlabel='Date'>
```



In []:

The Moving Average Crossover Strategy (Long - short)

we have two SMA filters the shorter lookback period SMA(50) and longer lookback period SMA(200) . We go long on TATAMOTORS at the first instance when SMA 50 exceeds SMA 200 , similarly we go short at the first instance when the SMA 200 exceeds the SMA 50.

RULES:

1. Go long on day $n+2$ when $SMA50 > SMA200$ on day $n+1$ and $SMA50 < SMA200$ on day n
2. Go short on day $n+2$ when $SMA50 < SMA200$ on day $n+1$ and $SMA50 > SMA200$ on day n

In [100... `## DMP_01_data_TATAMOTORS.NS`In [106... `df1 = pd.read_csv("DMP_01_data_TATAMOTORS.NS.csv" , index_col=0, parse_dates=True )`In [107... `df1`

Out[107...

	Open	High	Low	Close	Adj Close	Volume
Date						
2015-06-01	482.899994	482.899994	471.000000	472.049988	468.945648	4610219
2015-06-02	473.100006	478.399994	465.000000	466.950012	463.879272	6237231
2015-06-03	467.000000	468.750000	434.700012	456.850006	453.845642	9702740
2015-06-04	456.850006	458.450012	445.450012	451.200012	448.232819	8001049
2015-06-05	445.649994	449.549988	439.500000	442.100006	439.192657	11195670
...	...	...	...	...	...	...
2025-05-21	719.000000	729.500000	718.000000	726.799988	726.799988	8998596
2025-05-22	725.000000	725.000000	710.650024	717.450012	717.450012	10046820
2025-05-23	718.750000	721.799988	715.000000	718.250000	718.250000	7915166
2025-05-26	722.000000	738.000000	721.200012	729.000000	729.000000	13017400
2025-05-27	729.000000	729.349976	715.200012	716.450012	716.450012	12223125

2466 rows × 6 columns

In [110...

```

m = 50
n = 200
df1["sma50"] = df1["Adj Close"].rolling(window=m).mean()
df1["sma200"] = df1["Adj Close"].rolling(window=n).mean()

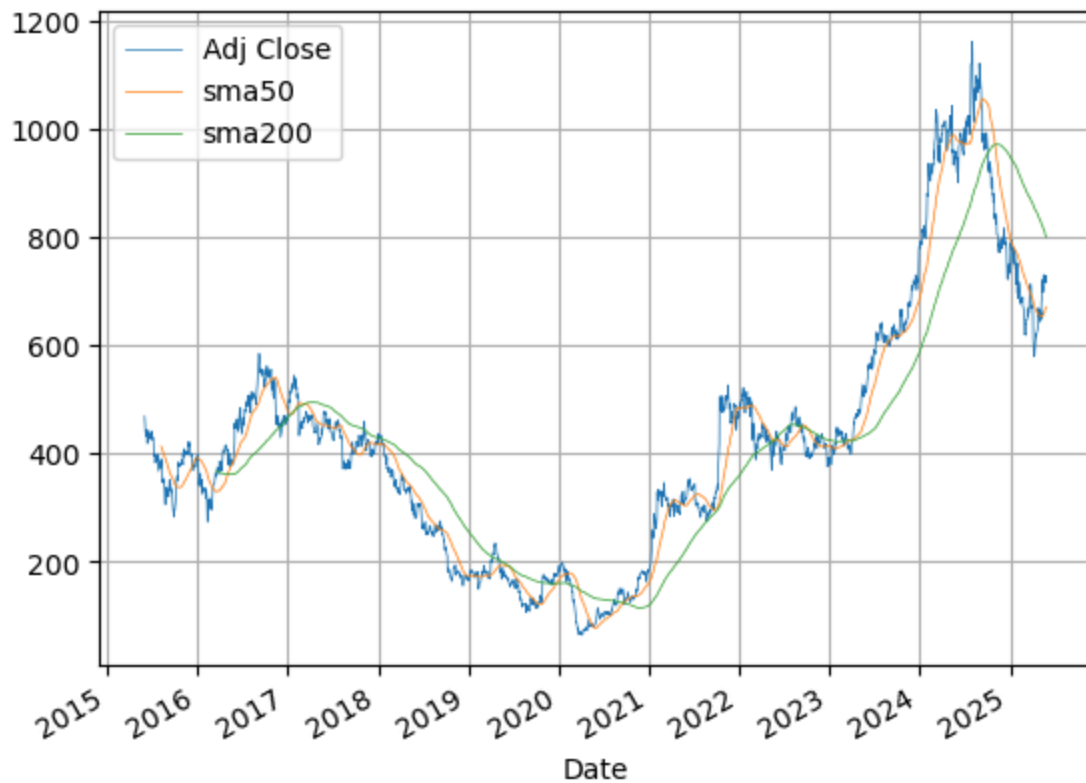
df1["sma50_prev_day"] = df1["sma50"].shift(1)
df1["sma200_prev_day"] = df1["sma200"].shift(1)

df1[["Adj Close", "sma50", "sma200"]].plot(grid=True, linewidth = 0.5)

```

Out[110...

<Axes: xlabel='Date'>



```
In [118...] df1["Buy"] = np.where((df1["sma50"] > df1["sma200"])
                        & (df1["sma50_prev_day"] < df1["sma200_prev_day"]), 1, 0)

df1["Sell"] = np.where((df1["sma50"] < df1["sma200"])
                       & (df1["sma50_prev_day"] > df1["sma200_prev_day"]), -1, 0)

In [122...] df1["trade_price"] = df1["Buy"] + df1["Sell"]
df1["position"] = df1["trade_price"].replace(to_replace=0, method="ffill")
df1
```

C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\151786233.py:2: FutureWarning: The 'method' keyword in Series.replace is deprecated and will be removed in a future version.

```
df1["position"] = df1["trade_price"].replace(to_replace=0, method="ffill")
```

Out[122...

	Open	High	Low	Close	Adj Close	Volume	sma50
Date							
2015-06-01	482.899994	482.899994	471.000000	472.049988	468.945648	4610219	NaN
2015-06-02	473.100006	478.399994	465.000000	466.950012	463.879272	6237231	NaN
2015-06-03	467.000000	468.750000	434.700012	456.850006	453.845642	9702740	NaN
2015-06-04	456.850006	458.450012	445.450012	451.200012	448.232819	8001049	NaN
2015-06-05	445.649994	449.549988	439.500000	442.100006	439.192657	11195670	NaN
...	...	...	...	...	...	...	...
2025-05-21	719.000000	729.500000	718.000000	726.799988	726.799988	8998596	665.067997
2025-05-22	725.000000	725.000000	710.650024	717.450012	717.450012	10046820	666.600997
2025-05-23	718.750000	721.799988	715.000000	718.250000	718.250000	7915166	668.172997
2025-05-26	722.000000	738.000000	721.200012	729.000000	729.000000	13017400	669.786997
2025-05-27	729.000000	729.349976	715.200012	716.450012	716.450012	12223125	671.152997

2466 rows × 15 columns

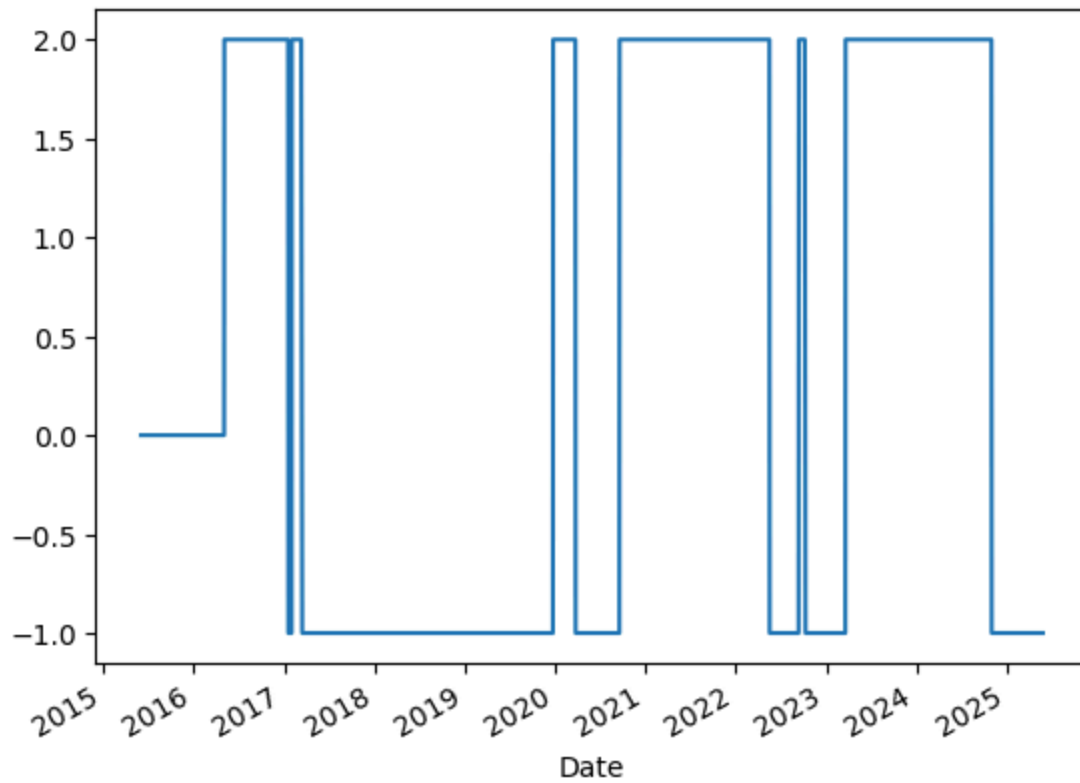


In [124...

```
df1["position"].plot()
```

Out[124...

<Axes: xlabel='Date'>

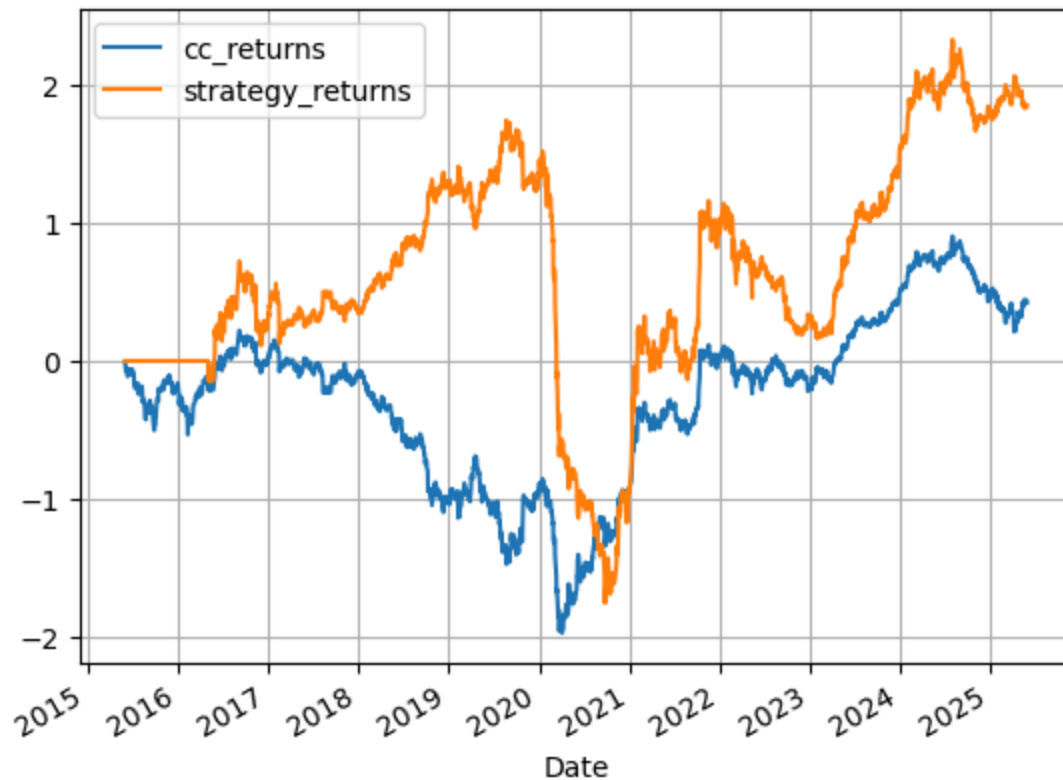


```
In [125...] df1["cc_returns"] = np.log(df1["Adj Close"]/df1["Adj Close"].shift(1))
```

```
In [127...] df1["strategy_returns"] = df1["cc_returns"] * df1["position"].shift(1)
```

```
In [129...] df1[["cc_returns", "strategy_returns"]].cumsum().plot(grid= True)
```

```
Out[129...] <Axes: xlabel='Date'>
```



In [132...

```
print("Buy and hold :", np.round(df1["cc_returns"].cumsum()[-1],2))
print("Strategy_returns:", np.round(df1["strategy_returns"].cumsum()[-1],2))
```

Buy and hold : 0.42

Strategy_returns: 1.86

C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\855523861.py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
print("Buy and hold :", np.round(df1["cc_returns"].cumsum()[-1],2))
```

C:\Users\Punith Kumar G L\AppData\Local\Temp\ipykernel_18584\855523861.py:2: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
print("Strategy_returns:", np.round(df1["strategy_returns"].cumsum()[-1],2))
```